



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

Web'in Temelleri: Ağdan HTTP'ye

Hafta 1

Web'in Temelleri: Ağdan HTTP'ye

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları:

1 • Ön-öğretim

2 • İçerik

3 • Katılım

4 • Değerlendirme

5 • İzleme

1 • Ön-öğretim Neden Önemli?

YG II'de bir `QTcpSocket` ile `connectToHost("127.0.0.1", 5000)` yazdığınızda IP adresi ve port sizin için zaten hazır veriliyordu.

- Peki tarayıcınıza yalnızca `www.google.com` yazdığınızda bu isim nereden bir IP adresine, bir de porta dönüşüyor?
- Sunucuya bağlandıktan sonra gönderdiğiniz baytlar aslında hangi “dili” konuşuyor?
- Bu hafta kod yazmadan, ağın en altından (kablo/Wi-Fi) başlayıp **HTTP**'ye kadar dört katmanı sırayla açacaksınız

YG II'de TCP soket temellerini ve Python'u biliyorsunuz — HTML/CSS/React/FastAPI'yi bu derste *ilk kez* göreceksiniz.

Bu Hafta

🎯 Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

- 1 TCP/IP katman modeli, IP adresi, port ve DNS'in rolünü açıklamak
- 2 Web sunucu mimarisi: istemci–sunucu, statik/dinamik, ters vekil, port 80/443
- 3 WebApp üç katmanlı mimarisi, API, SPA vs SSR farkı
- 4 HTTP isteği/cevabı anatomisi (curl, DevTools)
- 5 Katmanları Etkinlik/Bilet Sisteminin mimari haritasında yorumlama

BÖLÜM 1

Ünite 1 — TCP/IP Katmanları, IP Adresi, Port ve DNS

TCP/IP, Port, DNS Web Sunucu Mimarisi WebApp Mimarisi HTTP Anatomisi Özet

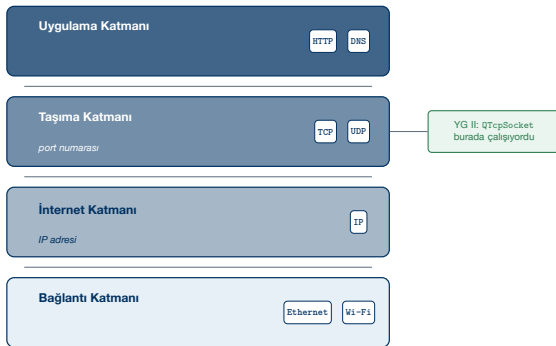
2 • İçerik Dört Katman, Sonra IP + Port + DNS

- **Uygulama katmanı** — HTTP, DNS
- **Taşıma katmanı** — TCP/UDP, **port numarası** (YG II'de `QTcpSocket` burada çalışıyordu)
- **İnternet katmanı** — IP protokolü, **IP adresi**
- **Bağlantı katmanı** — Ethernet/Wi-Fi

Tanım: IP adresi, port, DNS

IP adresi bir ağ arayüzünü tekil tanımlar; port aynı IP'deki servisleri ayırır (0–65535); DNS alan adını IP'ye çeviren dağıtık bir isim çözümleme sistemidir.

Dört Katmanlı TCP/IP Modeli



Basitleştirilmiş TCP/IP modeli (OSI'nin 7 katmanı bu derste kullanılmaz).

Analoji: Zarf Üstü Adres + Santral

Analoji

IP adresi + port'u bir zarf üstündeki adrese benzetin: posta kodu/cadde/ bina (IP adresi) mektubu doğru *binaya* götürür; daire numarası (port) aynı binadaki doğru *kapıya* götürür.

Analoji

DNS'i 118 numaralı bilgi/rehber santrale benzetin: bir adı bilip numarasını bilmediğinizde santrali ararsınız — santral `www.example.com` adını gerçek IP adresine çevirir.

Canlı Gözlem: dig ile DNS Çözümleme

```
1 # "dig +short" yalnızca cozümlenen IP adres(ler)ini basar (ANSWER bölümünün
2 # özeti); Windows'ta karşılığı "nslookup www.example.com" olup çıktı
3 # içindeki "Address:" satırı aynı IP'yi gösterir.
4 if ! command -v dig >/dev/null 2>&1; then
5     echo "HATA: 'dig' komutu bulunamadı. Windows'ta yerine 'nslookup ${ALAN_ADI}' çalıştırın." >&2
6 else
7     DIG_CIKTI=$(dig +short "${ALAN_ADI}" A 2>&1)
8     DIG_DURUM=$?
9     if [ ${DIG_DURUM} -ne 0 ] || [ -z "${DIG_CIKTI}" ]; then
10         echo "HATA: '${ALAN_ADI}' cozümlenemedi. Internet/DNS bağlantınızı kontrol edin." >&2
11     else
12         echo "${DIG_CIKTI}"
13     fi
14 fi
```

- Aynı betiğin `port_gozlemi` kesiti, `curl -v` çıktısında `Connected to ... port 443` satırını gösterir — DNS'in bulunduğu IP adresine hangi porttan bağlandığının kanıtıdır

BÖLÜM 2

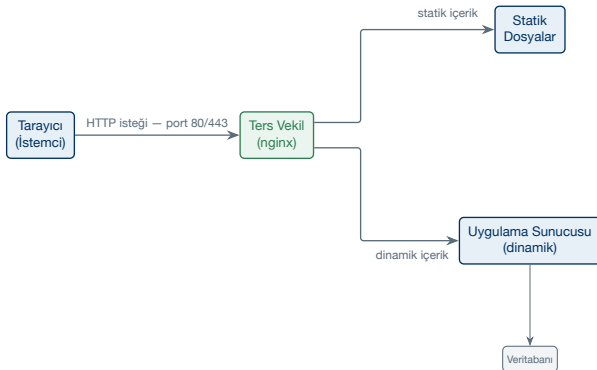
Ünite 2 — Standart Web Sunucu Mimarisi

TCP/IP, Port, DNS **Web Sunucu Mimarisi** WebApp Mimarisi HTTP Anatomisi Özet

2 • İçerik İstemci–Sunucu, Statik/Dinamik, Ters Vekil

- **İstemci–sunucu modeli:** istemci (tarayıcı/`curl`) isteği başlatır, sunucu bekler ve cevap verir
- **Statik içerik:** diskte hazır bir dosya, her istekte **aynı** cevap
- **Dinamik içerik:** sunucunun isteğe göre **anında ürettiği** cevap
- **Port 80/443 sözleşmesi:** HTTP → 80, HTTPS → 443 — teknik zorunluluk değil, evrensel bir sözleşme
- **Ters vekil:** istemciyle uygulama sunucusu arasına girip isteği yönlendiren aracı sunucu (ör. nginx)

Standart Web Sunucu Mimarisi



Ters vekil isteği statik dosyalara ya da dinamik uygulama sunucusuna yönlendirir; Hf.14'te Docker Compose ile fiilen kurulacak.

Kod: Statik vs Dinamik İstek

```
1 # example.com sunucudaki hazır bir HTML dosyasını olduğu gibi gönderir; iki
2 # ardisik istek birebir aynı govdeyi dondurmeliştir.
3 ISTEK1=$(curl -sS -m 10 "${STATIK_URL}")
4 DURUM1=$?
5 ISTEK2=$(curl -sS -m 10 "${STATIK_URL}")
6 DURUM2=$?
7
8 if [ ${DURUM1} -ne 0 ] || [ ${DURUM2} -ne 0 ]; then
9     echo "HATA: ${STATIK_URL} adresine bağlanılamadı. İnternet bağlantınızı kontrol edin."
10    >&2
11 elif [ "${ISTEK1}" = "${ISTEK2}" ]; then
12     echo "Statik cevap iki istekte de BİREBİR AYNI (beklenen davranış)."
```

- Aynı betiğin `dinamik_istek` kesiti `httpbin.org/uuid`'i iki kez ister — her seferinde **Farklı** bir kimlik döner

BÖLÜM 3

Ünite 3 — Üç Katman, API, SPA vs SSR

TCP/IP, Port, DNS Web Sunucu Mimarisi **WebApp Mimarisi** HTTP Anatomisi Özet

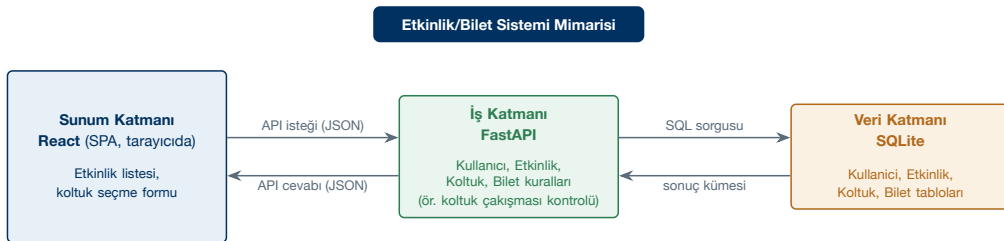
2 • İçerik Üç Katmanlı Mimari

Bir bilet satış sisteminde: koltuğa tıklama, boşluk kontrolü, kayıt — bu üç işi ayrı katmanlara bölersiniz.

- **Sunum katmanı:** kullanıcının gördüğü/etkileşime girdiği arayüz
- **İş katmanı:** kuralların uygulandığı mantık (ör. “koltuk iki kez satılamaz”)
- **Veri katmanı:** bilginin kalıcı tutulduğu yer

YG I/II'nin Qt+SQLite masaüstü uygulamasında bu üçü **aynı süreçteydi**; web bunları ayırıştırır.

Bu Dersin Yığını: React + FastAPI + SQLite



Etkinlik/Bilet Sisteminin dönem boyu referans haritası.

Analoji: Garson, Mutfak, Kiler + Sipariş Fişi

Analoji

Üç katmanı bir restorana benzetin: garson (sunum) siparişi alır ama pişirmez; mutfak (iş) tarifi uygular ama malzeme depolamaz; kiler (veri) malzemeyi saklar ama pişirmez.

Analoji

API'yi sipariş fişine benzetin: garson mutfağa doğrudan girmez, standart bir fiş doldurup iletir — fişin biçimi önceden anlaşılmıştır.

SPA vs SSR

Tanım: SPA / SSR

SPA: tarayıcı bir kez HTML/JS indirir, sonra yalnız API'den veri çekip arayüzü JavaScript ile günceller. **SSR:** her sayfa değişiminde sunucu tam bir HTML sayfası üretip gönderir.

Analoji

Bir kitapçıkla gezmek (SPA) ile her soruda yeni broşür almak (SSR) gibi düşünün.

SPA yaklaşımını (React + Vite) izleyeceksiniz; SSR'ı yalnız kavram düzeyinde tanıyacak, bu derste uygulamayacaksınız.

BÖLÜM 4

Ünite 4 — HTTP İstek/Cevap Anatomisi

TCP/IP, Port, DNS Web Sunucu Mimarisi WebApp Mimarisi **HTTP Anatomisi** Özet

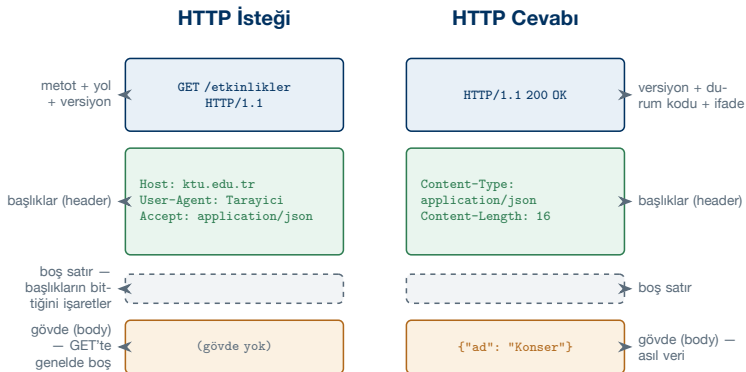
2 • İçerik HTTP Metotları ve Durum Kodu Aileleri

- GET oku, POST oluştur, PUT/PATCH güncelle, DELETE sil
- **2xx** başarılı, **3xx** yönlendirme, **4xx** istemci hatası, **5xx** sunucu hatası

Analoji

Durum kodunu gişe memurunun kısa cevabına benzetin: “Tamam, alın” (2xx), “Böyle bir belge yok” (4xx), “Şu an sistemimiz arızalı” (5xx).

HTTP İstek/Cevap Anatomisi



İstek satırı/durum satırı, başlıklar, boş satır, gövde — her ikisi de aynı dört parçalı yapıyı izler.

Ham Bir İstek ve Cevap

>_ Terminal

```
GET /etkinlikler HTTP/1.1  
Host: biletsistemi.example.com  
User-Agent: curl/8.4.0  
Accept: application/json
```

🖥 Beklenen Çıktı

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Content-Length: 52  
  
{ "etkinlik": "KTU Bahar Senligi", "kalanKoltuk": 42 }
```

curl -v: Giden/Gelen Başlıklar

```
1 # "> " ile baslayan satirlar giden istegi (istek satiri + basliklar), "< "  
2 # ile baslayan satirlar gelen cevabi (durum satiri + basliklar) gosterir.  
3 GET_CIKTI=$(curl -sS -v -m 10 --http1.1 "https://httpbin.org/get" 2>&1)  
4 GET_DURUM=$?  
5 if [ ${GET_DURUM} -ne 0 ]; then  
6     echo "HATA: httpbin.org/get adresine baglanilamadi. Internet baglantinizi  
7     kontrol edin." >&2  
8 else  
9     echo "${GET_CIKTI}" | grep -E "^(> |< )"fi
```

- > giden isteğiniz, < sunucudan gelen cevap
- Aynı betiğin `post_istek_govde` kesiti, JSON gövdeli bir `POST`'un httpbin tarafından aynen yansıtıldığını gösterir

HTTP Aslında Düz Metin: Ham Söket ile GET I

```
1 istek = (  
2     f"GET / HTTP/1.1\r\n"  
3     f"Host: {HOST}\r\n"  
4     f"User-Agent: hafta01-söket-istemci\r\n"  
5     f"Accept: text/html\r\n"  
6     f"Connection: close\r\n"  
7     f"\r\n"  
8 )  
9  
10 try:  
11     with socket.create_connection((HOST, PORT), timeout=10) as soket:  
12         soket.sendall(istek.encode("ascii"))  
13         cevap = b""  
14         while True:  
15             parca = soket.recv(4096)  
16             if not parca:  
17                 break
```

HTTP Aslında Düz Metin: Ham Soket ile GET II

```
18         cevap += parca
19     print(cevap.decode("utf-8", errors="replace"))
20 except OSError as hata:
21     print(
22         f"HATA: {HOST}:{PORT} adresine baglanilamadi ({hata}). "
23         "Internet baglantinizi kontrol edin.",
24         file=sys.stderr,
25     )
26     sys.exit(1)
```

`socket.create_connection` $\equiv$ YG II'deki `connectToHost`; `sendall/recv` ham HTTP metnini gönderir/alır.

BÖLÜM 5

Özet

TCP/IP, Port, DNS Web Sunucu Mimarisi WebApp Mimarisi HTTP Anatomisi **Özet**

Özet

Özet

- TCP/IP'nin dört katmanı + IP/port + DNS: doğru makineye, doğru servise, doğru isimle ulaşırsınız
- İstemci–sunucu modeli, statik/dinamik içerik, port 80/443 ve ters vekil: bir web sunucusunun standart mimarisi
- Üç katmanlı mimari (sunum/iş/veri) + API + SPA/SSR: bu dersin yığını (React/FastAPI/SQLite) bu şemaya yerleşir
- HTTP isteği/cevabı hep aynı dört parçadan oluşur: satır, başlıklar, boş satır, (varsa) gövde — ve bu aslında düz metindir

5 • İzleme Aktarım — Başka Nerede?

- Bu hafta öğrendiğiniz IP+port ikilisi, Hf.6'da `uvicorn app.main:app --port 8000` ile FastAPI sunucunuzu ayağa kaldırdığınızda somutlaşacak
- Hf.14'te bu haftanın kavram olarak öğrendiği statik/dinamik ayrımı ve ters vekil, gerçek bir `docker compose` yapılandırmasına dönüşecek
- webapp - uc - katman şeması dönem boyu geri döneceğiniz harita: Hf.4-5 sunum katmanı, Hf.6 iş katmanı, Hf.8 veri katmanı
- `soket-istemci.py`'nin elle yazdığı anatomi, Hf.6'da FastAPI'nin sizin yerinize ürettiği cevapla **birebir aynı** kalıptır

Sırada Ne Var?

Bu hafta lab'da

`curl` ile gerçek bir sunucuya (`https://httpbin.org/get`) ilk isteklerinizi göndürecek, Etkinlik/Bilet Sisteminin üç katmanlı mimari şemasını inceleyeceksiniz — henüz kod yazmayacaksınız.

- **Kendinizi Sınavın** her üniteyi kapsıyor: IP+port+DNS birlikte ne iş yapar, ters vekil neden araya girer, koltuk kuralı hangi katmanda, **GET/POST** gövdesi neden farklı
- Gelecek hafta: **HTML5 ve CSS3** — bu haftanın “sunum katmanı” kavramına ilk somut malzemeyi, statik bir Etkinlik/Bilet sayfasını, bizzat yazacaksınız